Common Core: Phase 2 Verification

Lab V2: Verifying a Shared Resource

Virtual Interfaces, Dynamic Arrays, fork/join and Protocol Assertions

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Verify the dual-master arbitrated memory of Lab R3 with a testbench that drives both masters in parallel through virtual interfaces, keeps stimulus and expected data in dynamic arrays, queues and associative arrays, checks results in the order the DUT accepted them, and adds protocol assertions to `mem_if` and arbiter assertions bound from `dv/sva/`.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Labs R3 and V1
Designs	`arb_mem`, `mem_if` (verified; protocol assertions added to `mem_if`; RTL otherwise unchanged unless a bug is found)
Reused in	V3 (`arb_mem` is the cache's backing memory; process and scoreboard patterns), T-DV1 and T-DV2 (`mem_if` drivers, monitors and protocol assertions), P3 (`AW` configurations)

Contents

1 Overview **3**

2 The Design Under Test **3**

3 Virtual Interfaces **4**

4 Testbench Arrays **5**

4.1 Dynamic Array Operations 5

4.2 Queue and Associative Array Operations 6

5 Concurrent Processes: `fork` and `join` **6**

5.1 Typical Patterns 6

5.2 The Loop Variable Pitfall 7

5.3 Shared Data Between Processes 7

6 Testbench Architecture **8**

6.1 Structure 8

6.2 Driver 9

6.3 Scoreboard in Acceptance Order 9

6.4 Ending the Test and Statistics 10
6.5 Tests and Plusargs 11

7 Assertions **12**
7.1 Protocol Assertions in `mem_if` 12
7.2 Arbiter Assertions, Bound from `dv/sva/` 13
7.3 Cover Properties 13
7.4 Exercise: Assertion Fires First 14

8 Clocking Blocks (Optional) **14**

9 Testplan and Regression **15**

10 Requirements / Tasks **15**

11 Repository and Flow **16**

12 Deliverables **16**

13 Assessment **17**

14 Review Questions **17**

15 References **18**

1 Overview

A processor core shares resources all the time: the instruction fetch unit and the load/store unit both access memory, and they ask at the same time. In Lab R3 you built a small version of that situation, `arb_mem`: a memory with two `mem_if` master ports and a round-robin arbiter. Its smoke test drove both masters from one sequential `initial` block, which can only create the conflicts you thought of in advance.

Real masters do not take turns. Verifying a shared resource needs a testbench whose masters run as independent processes, issue requests whenever they want, and are checked by a scoreboard that does not know in advance who will win each conflict — the DUT decides the order, and the testbench must follow it. This lab builds that testbench with three language features: **virtual interfaces** (one driver task serves any port), **testbench arrays** (dynamic arrays, queues and associative arrays for stimulus, expected data and statistics) and **`fork/join`** (parallel drivers, monitors and a watchdog). The protocol rules of `mem_if`, specified in Lab R3, are now written as assertions inside the interface, so every instance checks itself.

By the end of this lab you should be able to:

1. Pass a `virtual interface` to a task, so one driver or monitor task serves any number of ports.
2. Choose between dynamic arrays, queues and associative arrays for a testbench job, and use their methods without the classic pitfalls.
3. Use `fork/join`, `join_any`, `join_none`, `wait fork` and `disable fork` correctly, including inside loops.
4. Build a scoreboard for a shared resource where the order of operations is decided by the DUT, not by the testbench.
5. Write protocol assertions inside an interface and design assertions for an arbiter bound from `dv/sva/`, with cover properties that prove the conflicts happened.
6. Explain what a clocking block does and how it relates to the `<=` drive standard.

Where this lab sits. It verifies the RTL of Lab R3 with the environment structure, `tb_utils_pkg`, plusargs and `make regress` from Lab V1, unchanged. Lab V3 reuses `arb_mem` as the backing memory of the cache and the process and scoreboard patterns of this lab; Labs T-DV1 and T-DV2 turn the `mem_if` driver and monitor into reusable class-based and UVM components, and keep the protocol assertions written here.

2 The Design Under Test

The full specification is in Lab R3 and in `hw/ip/arb_mem/doc/spec.md`. The rules that the testbench and the assertions of this lab check are repeated here.

Signal	Driven by	Width	Description
`req_valid`	master	1	A request is present
`req_ready`	slave	1	The request is accepted at this edge
`req_we`	master	1	1 = write, 0 = read
`req_addr`	master	`AW`	Word address
`req_wdata`	master	`DW`	Write data
`rsp_valid`	slave	1	Read data is valid
`rsp_rdata`	slave	`DW`	Read data

Protocol (per port):

P1. A request is *accepted* at a rising edge where `req_valid && req_ready`.

P2. While `req_valid && !req_ready`, the master holds `req_valid`, `req_we`, `req_addr` and `req_wdata` stable.

P3. `req_valid` must not depend on `req_ready`; `req_ready` may depend combinationally on `req_valid`.

P4. An accepted write updates the memory at the accepting edge. Writes have no response.

P5. An accepted read produces `rsp_valid = 1` for exactly one cycle, exactly one cycle after acceptance, with the data of that address including every write accepted at an earlier edge.

Arbitration:

A1. At most one request (from either master) is accepted per cycle.

A2. If only one master requests, it is accepted in the same cycle.

A3. If both request, the master that was *not* granted most recently wins (round robin); a waiting master is accepted after at most one cycle of waiting.

A4. After reset, master 0 has priority.

Parameters: `AW` (default 4, `DEPTH = 2**AW`) and `DW` (default 32). Lab R3 required the internal request and grant vectors to be packed arrays `logic [1:0] req, gnt`; the arbiter assertions of this lab use them.

3 Virtual Interfaces

An interface instance is a piece of hierarchy, not a variable, so it cannot be passed to a task directly. A **virtual interface** is a handle (a reference) to an interface instance. Tasks receive handles, so one driver task can drive any port, and the task no longer needs to be compiled inside the module that declares the signals:

```
module tb_top #(parameter int AW = 4, parameter int DW = 32);
  logic clk = 1'b0, rst_n;
  initial forever #5 clk = ~clk;

  mem_if #(AW, DW) m_if [2] (.clk(clk), .rst_n(rst_n)); // array of instances
  virtual mem_if #(AW, DW) vif [2];                     // array of handles

  initial begin
    vif[0] = m_if[0];        // constant index: instance arrays are hierarchy,
    vif[1] = m_if[1];        // a variable index (m_if[i]) is illegal here
  end

  arb_mem #(.AW(AW), .DW(DW)) dut (.clk, .rst_n, .m0(m_if[0]), .m1(m_if[1]));
  // ...
endmodule
```

Points to remember:

- The parameters of the virtual interface type must match those of the instance (`virtual mem_if #(AW, DW)`); a mismatch is a compile error, not a silent conversion.
- A handle that was never assigned is `null`; using it is a run-time error. Assign handles before any process uses them (time 0, before reset).

- The interface instances, not the handles, are connected to the DUT. Handles are only for procedural code.

This is the same mechanism a UVM driver uses to reach the DUT (Lab T-DV2); here you use it from plain tasks. In Lab V1 the tasks were `` `included `` into `tb_top` because they used its signals by name; now the driver and monitor tasks reach the DUT only through their `vif` argument.

4 Testbench Arrays

Lab R3 used the two synthesizable kinds of arrays: packed arrays for control vectors and a static unpacked array for the memory. A testbench needs collections whose size is decided at run time.

Kind	Declaration	Properties	Synth.
Packed	`logic [7:0] b;`	Contiguous bits; usable as one number in arithmetic	Yes
Unpacked, static	`logic [7:0] m [16];`	Fixed number of separate elements; size known at compile time	Yes
Dynamic	`int d [];`	Size set at run time with `new[n]`; contiguous	No
Queue	`int q [$];`	Grows and shrinks at both ends; `push_back`, `pop_front`	No
Associative	`int a [int];`	Sparse; only written indices exist; `exists()`, `delete()`	No

How to choose in the testbench:

- **Dynamic array:** a collection whose size is known before it is filled, but only at run time — for example a sequence of `NUM_TXN` transactions, where `NUM_TXN` comes from a plusarg.
- **Queue:** first-in first-out traffic whose length changes during the run — for example expected read data waiting for its response.
- **Associative array:** a large, sparsely used index space — for example a reference model of a memory where only a few addresses are written, or a histogram of latencies.

4.1 Dynamic Array Operations

```
typedef struct {
  bit            we;
  logic [AW-1:0] addr;
  logic [DW-1:0] wdata;
  int unsigned   idle;   // idle cycles before this request
} txn_t;

txn_t seq [];                     // empty, size 0
seq = new[num_txn];               // allocate num_txn elements (default values)
seq = new[seq.size() + 10](seq); // grow by 10 and KEEP the old contents
seq = new[20];                    // reallocate: old contents are LOST
foreach (seq[i]) seq[i].addr = $urandom_range(addr_range-1);
$display("size = %0d", seq.size());
seq.delete();                     // size 0 again

int unsigned lat [];              // per-transaction latency
longint total = lat.sum() with (longint'(item));  // widen to avoid overflow
int unsigned mx [$] = lat.max(); // locator methods return a QUEUE
```

4.2 Queue and Associative Array Operations

```
logic [DW-1:0] q [$];
q.push_back(x);   q.push_front(y);     // add at either end
x = q.pop_front(); y = q.pop_back();   // remove (empty queue: warning + default)
x = q[0];  y = q[$];                   // first and last element, not removed
q.insert(2, z);   q.delete(2);  q.delete();   // by index; all

logic [DW-1:0] ref_mem [logic [AW-1:0]];  // index type = address type
ref_mem[addr] = data;                     // creates the entry
if (ref_mem.exists(addr)) data = ref_mem[addr];
$display("%0d addresses written", ref_mem.num());
foreach (ref_mem[a]) $display("%h: %h", a, ref_mem[a]);  // ascending index order
ref_mem.delete(addr);  ref_mem.delete();
```

Pitfalls to know:

- `new[n]` without `(old)` discards the old contents.
- Reading an index outside a dynamic array, popping an empty queue or reading a non-existent associative entry returns the default value of the element type and a simulator warning, not an error. Check sizes and `exists()` explicitly; a scoreboard that compares against a silent default can pass by accident.
- Growing a dynamic array element by element (`new[size+1]`) copies the whole array each time. If the final size is unknown, use a queue.
- `max()`, `min()` and `unique()` return a queue, not a single value. `sum()` returns the element type: the sum of many `int` latencies can overflow without `with (longint'(item))`.

5 Concurrent Processes: `fork` and `join`

Two masters issue requests whenever they want, including in the same cycle. The testbench must therefore run their drivers as independent processes.

Construct	The parent process continues when
`fork ... join`	all child processes have finished
`fork ... join_any`	any one child process has finished
`fork ... join_none`	immediately; the children keep running
`wait fork`	all children started by this process have finished
`disable fork`	— (kills all children started by this process)

5.1 Typical Patterns

```
// 1. Run both masters in parallel and wait for both
fork
  run_master(vif[0], seq0);
  run_master(vif[1], seq1);
join

// 2. Background processes that never end
fork
  mon_req(vif[0], 0);  mon_rsp(vif[0], 0);
  mon_req(vif[1], 1);  mon_rsp(vif[1], 1);
```

```
join_none

// 3. Watchdog: whichever finishes first wins
fork
  begin run_all_tests(); end
  begin repeat (max_cycles) @(posedge clk);
        $error("TIMEOUT"); end
join_any
disable fork;   // kill the loser (and the background monitors!)
```

`disable fork` kills *all* child processes of the current process, including monitors started earlier with `join_none`. Wrap the watchdog pattern in its own `fork begin ... end join` if the monitors must survive:

```
fork begin                        // isolating thread
  fork
    fork run_master(vif[0], seq0); run_master(vif[1], seq1); join
    begin repeat (max_cycles) @(posedge clk); check(0, "TIMEOUT"); end
  join_any
  disable fork;                   // kills only the children of this thread
end join
```

5.2 The Loop Variable Pitfall

```
// BAD: all children see the final value of i (2), after the loop ends
for (int i = 0; i < 2; i++)
  fork
    run_master(vif[i], seqs[i]);
  join_none

// GOOD: an automatic variable captures the value for each child
for (int i = 0; i < 2; i++)
  fork
    automatic int k = i;
    run_master(vif[k], seqs[k]);
  join_none
wait fork;
```

`join_none` only *schedules* the child; it starts running when the parent blocks or finishes the time step. By then the loop has ended and `i` is 2. The automatic variable declared inside the fork is initialized when the fork statement executes, so each child gets its own copy.

5.3 Shared Data Between Processes

When two processes write the same testbench variable in the same time step, the result depends on their order: the race of Lab R2, now inside the testbench. In this lab both request monitors update one reference model. This is safe only because the DUT accepts at most one request per cycle (rule A1) — which is exactly what one of your assertions checks. If that assertion ever fires, the scoreboard results of the same run are not trustworthy. Explain this dependency in `doc/verification.md`.

The expected-data queues are shared too: at one edge the request monitor may `push_back` a new expectation while the response monitor `pop_front`s an older one. That is order-independent, because a queue with one producer at the back and one consumer at the front gives the same result in either order.

NOTE | Tool note.

Verilator 5 with `--timing` supports `fork/join`, `join_any`, `join_none`, `wait fork` and `disable fork`; very old 5.x versions lack some of them. Run the `race_demo`-style check of Lab R2 if a result looks order-dependent: Verilator orders processes at compile time, so a testbench race may be stable in Verilator and still be a race.

SYNOPSYS CLOUD

VCS (`make sim SIM=vcs`, the cloud default) supports every construct of this lab: virtual interfaces and arrays of interface instances, all `fork` variants, queues and associative-array methods, concurrent assertions with `##` sequences and `default disable iff`, and cover properties with pass actions. Verilator 5.013 on the cloud (below the program minimum 5.020) cannot build this testbench: it rejects `##` in properties and `default disable iff` (`Error-UNSUPPORTED`), and it executes the `<=` drives of tasks called from `initial` blocks as blocking assignments (warning `INITIALDLY`), which breaks the drive standard. Use `SIM=vcs` there. VCS is four-state: reading a never-written address returns X, which is why the scoreboard counts such reads as "unknown" instead of comparing them.

6 Testbench Architecture

6.1 Structure

Figure 1 shows the whole environment. The two drivers and the four monitors reach the DUT only through virtual interface handles; the protocol assertions live inside each `mem_if` instance, and the arbiter assertions are attached to `arb_mem` with `bind`.

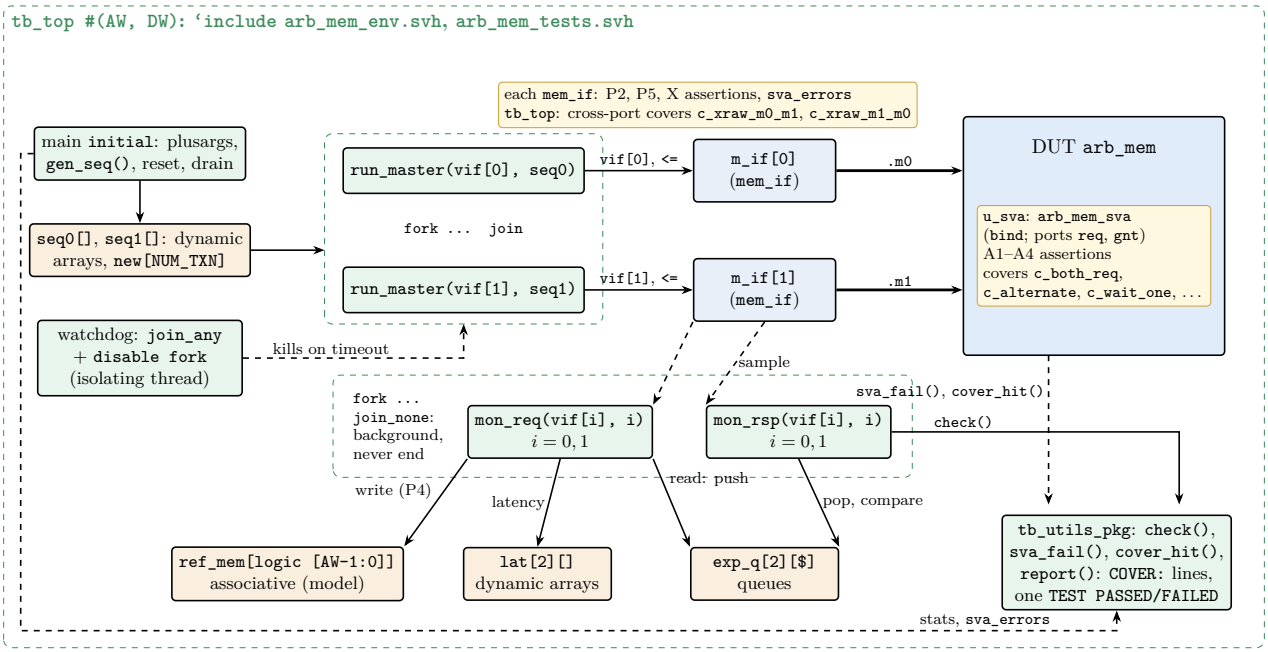


Figure 1: Lab V2 testbench architecture. Green: testbench processes and code; orange: testbench arrays; yellow: assertions and cover properties (simulation only). Solid thin arrows: drive or data flow; dashed: sampling, reporting or process control.

Component	Implemented as	Responsibility
Sequence generator	`function automatic void gen_seq(ref txn_t seq[], input int n, input int id)`	Allocate the dynamic array with `new[n]` and fill it according to the test
Driver	`task automatic run_master(virtual mem_if #(AW,DW) vif, ref txn_t seq[])`	Drive every transaction with `<=`, honoring idle cycles and the handshake
Request monitor	`task automatic mon_req(virtual mem_if #(AW,DW) vif, input int id)`	On every accepted request: apply writes to the reference model; for reads, push the expected data into `exp_q[id]`; record the acceptance latency
Response monitor	`task automatic mon_rsp(virtual mem_if #(AW,DW) vif, input int id)`	On every `rsp_valid`: pop `exp_q[id]` and compare
Reference model	Associative array `logic [DW-1:0] ref_mem [logic [AW-1:0]]`	Memory contents as written so far
Statistics	Dynamic arrays `int unsigned lat [2][]`	Cycles from `req_valid` to acceptance, per transaction and master
Watchdog	`fork ... join_any` (or `tb_utils_pkg::watchdog`)	End the test with an error if it hangs

Files: the tasks live in `dv/env/arb_mem_env.svh`, the tests in `dv/tests/arb_mem_tests.svh`, both included into `dv/tb/tb_top.sv`; `tb_utils_pkg` from Lab V1 provides `check`, `report`, the watchdog and the plusarg helpers.

6.2 Driver

```
// Precondition: called right after a rising edge, req_valid low.
task automatic run_master(virtual mem_if #(AW, DW) vif, ref txn_t seq[]);
  foreach (seq[i]) begin
    repeat (seq[i].idle) @(posedge vif.clk);
    vif.req_valid <= 1'b1;
    vif.req_we    <= seq[i].we;
    vif.req_addr  <= seq[i].addr;
    vif.req_wdata <= seq[i].wdata;
    do @(posedge vif.clk); while (!vif.req_ready);   // accepted here
    vif.req_valid <= 1'b0;
  end
endtask
```

The driver honors rule P2 by construction: it changes nothing while waiting. Rule P3 also holds: `req_valid` is decided before `req_ready` is looked at.

6.3 Scoreboard in Acceptance Order

The two masters' sequences are generated independently, but the *order* in which their requests reach the memory is decided by the arbiter. A scoreboard that predicted read data from the generated order would be wrong whenever a write of one master and a read of the other target the same address. The reference model is therefore updated by the request monitors, in the order the DUT *accepted* the requests:

```
typedef struct { bit known; logic [DW-1:0] data; } exp_t;
exp_t          exp_q [2][$];                 // one queue per master
logic [DW-1:0] ref_mem [logic [AW-1:0]];     // reference model
int unsigned   lat [2][];                    // allocated with new[num_txn]
```

```
int unsigned   n_acc [2], n_rd [2], n_wr [2], n_unknown [2];

task automatic mon_req(virtual mem_if #(AW, DW) vif, input int id);
  int unsigned wait_cyc = 0;
  forever begin
    @(posedge vif.clk);
    if (!vif.rst_n || !vif.req_valid) continue;
    if (!vif.req_ready) begin wait_cyc++; continue; end   // waiting
    if (n_acc[id] < lat[id].size()) lat[id][n_acc[id]] = wait_cyc;
    n_acc[id]++;
    wait_cyc = 0;
    if (vif.req_we) begin
      ref_mem[vif.req_addr] = vif.req_wdata;              // P4
      n_wr[id]++;
    end else begin
      exp_q[id].push_back(ref_mem.exists(vif.req_addr)
          ? exp_t'{1'b1, ref_mem[vif.req_addr]} : exp_t'{1'b0, '0});
      n_rd[id]++;
    end
  end
endtask

task automatic mon_rsp(virtual mem_if #(AW, DW) vif, input int id);
  exp_t e;
  forever begin
    @(posedge vif.clk);
    if (!vif.rst_n || !vif.rsp_valid) continue;
    if (exp_q[id].size() == 0) begin
      check(1'b0, $sformatf("m%0d: response without read", id));
      continue;
    end
    e = exp_q[id].pop_front();
    if (e.known) check(vif.rsp_rdata === e.data,
        $sformatf("m%0d: rdata %h, expected %h", id, vif.rsp_rdata, e.data));
    else n_unknown[id]++;          // never written: counted, not compared
  end
endtask
```

A read of an address that was never written is not compared (the memory is not reset), but it is counted and reported; a test in which most reads are “unknown” checks little and must be fixed.

6.4 Ending the Test and Statistics

The test ends when both drivers have finished, the last response has arrived and both `exp_q` queues are empty — or when the watchdog fires. At the end, report per master: number of reads, writes, checks, errors, unknown reads, and the minimum, average and maximum acceptance latency, computed with the array methods:

```
function automatic void print_stats();
  for (int id = 0; id < 2; id++) begin
    int unsigned mn [$], mx [$];
    int unsigned hist [int unsigned];          // latency histogram
    mn = lat[id].min();  mx = lat[id].max();
    foreach (lat[id][i]) hist[lat[id][i]]++;
    $display("STATS m%0d: rd=%0d wr=%0d unknown=%0d lat min/avg/max=%0d/%0.2f/%0d",
             id, n_rd[id], n_wr[id], n_unknown[id], mn[0],
             real'(lat[id].sum() with (longint'(item))) / lat[id].size(), mx[0]);
    foreach (hist[l]) $display("  m%0d latency %0d: %0d", id, l, hist[l]);
```

```
  end
endfunction
```

Check that `n_acc[id]` equals the number of generated transactions: a request that was generated but never accepted is a hang the watchdog should already have reported. By rule A3 the maximum latency must be 1; the statistics are a second, independent check of fairness.

6.5 Tests and Plusargs

TEST TERM | Random test

Stimulus drawn from `$urandom` with the printed seed (`+SEED=`), thousands of transactions, checked by the reference model and the scoreboard. It creates conflicts and read-after-write sequences nobody wrote down; the same seed reproduces a failure exactly. All four tests below are random tests with different shapes; `random` is the least constrained one.

TEST TERM | Stress / back-pressure test

Stress: both sides run at the maximum rate. Back-pressure: the consumer holds `ready` low and the producer must wait. In this lab the arbiter is the source of back-pressure: the losing master sees `req_ready = 0` and must hold its request (P2). `conflict` is the stress test (both masters request in every cycle, so every cycle is a conflict and every request except the first waits exactly one cycle); `random` applies back-pressure at random moments.

+TEST=	Scenario	What a pass shows
`single`	Only master 0 is active, then only master 1	Each port and the data path alone; a lone requester is never delayed (A2, latency 0)
`write_read`	Each master writes a region, then reads it back; regions do not overlap	Data integrity of both ports in parallel, without cross-master address sharing
`conflict`	Both masters request in every cycle (`IDLE_PCT=0`) on the same few addresses	Stress: alternating grants, latency exactly 1 for the waiting master (A3), read-after-write across masters (P4, P5)
`random`	Both masters, random mix of reads and writes, random idle cycles	Everything at once, with conflicts at random moments; the long-run check

Plusarg	Default	Meaning
`+NUM_TXN=`	1000	Transactions per master (size of each dynamic array)
`+WR_PCT=`	50	Percentage of writes
`+IDLE_PCT=`	30	Probability of idle cycles between requests
`+ADDR_RANGE=`	`DEPTH`	Addresses used are `0 .. ADDR_RANGE-1`; a small value forces address conflicts
`+MAX_ERRORS=`	20	Stop after this many errors (`tb_utils_pkg`); raise it to see a whole failing run

The Lab V1 rules remain: drives with `<=`, plusargs with defaults printed in the `CONFIG:` line, `$urandom` and `$urandom_range` only, reproducible seeds, unknown test names are errors, exactly one `TEST PASSED` or `TEST FAILED` line.

7 Assertions

TEST TERM | Assertion (SVA) checks

Properties checked at every clock edge during every test. A scoreboard compares *results* and notices a bug only when a wrong value reaches it, possibly thousands of cycles later; an assertion reports a protocol or design-rule violation in the cycle it happens, with the name of the rule. They also check behavior that never shows up in the data, such as fairness.

7.1 Protocol Assertions in `mem_if`

Protocol assertions belong in the interface: every instance of `mem_if` then checks its own port automatically — in this testbench, in the cache of Lab R4 that uses `arb_mem`, and in any future design. They are added to `hw/ip/arb_mem/rtl/mem_if.sv` inside `` `ifndef SYNTHESIS ``, as Lab R3 anticipated.

```
`ifndef SYNTHESIS
  int unsigned sva_errors = 0;          // read by the testbench before its verdict
  function automatic void sva_fail(input string label);
    sva_errors++;
    $display("ERROR: [%0t] ASSERTION FAILED: %m.%s", $time, label);
  endfunction
  // P2: request held stable while waiting
  a_req_stable: assert property (@(posedge clk) disable iff (!rst_n)
    req_valid && !req_ready |=>
      req_valid && $stable({req_we, req_addr, req_wdata}))
    else sva_fail("a_req_stable");
  // P5: every accepted read is answered exactly one cycle later
  a_rsp_after_read: assert property (@(posedge clk) disable iff (!rst_n)
    req_valid && req_ready && !req_we |=> rsp_valid)
    else sva_fail("a_rsp_after_read");
  // P5: no response without a read accepted in the previous cycle
  a_no_spurious_rsp: assert property (@(posedge clk) disable iff (!rst_n)
    rsp_valid |-> $past(req_valid && req_ready && !req_we))
    else sva_fail("a_no_spurious_rsp");
  // no X on control signals after reset (four-state simulators)
  a_ctrl_known: assert property (@(posedge clk) disable iff (!rst_n)
    !$isunknown({req_valid, req_ready, rsp_valid}))
    else sva_fail("a_ctrl_known");
`endif
```

Note *whom* each assertion checks. `a_req_stable` is an obligation of the master: in this testbench it checks your driver. In a formal tool it would be written as `assume property` for the slave's proof and `assert property` for the master's. The response assertions are obligations of the slave, `arb_mem`.

Making a failure fail the test. Without an action block, VCS prints an assertion failure and the test still ends with `TEST PASSED`. The checker modules in `dv/sva/` call `tb_utils_pkg::sva_fail()`, but `mem_if` is RTL: it is also compiled by the Lab R3 smoke test and by the cache of Lab R4, which do not import the testbench package. The interface therefore counts its own failures in `sva_errors`, and every testbench adds `vif[i].sva_errors` to its error count before the verdict. The action block prints with `$display`, not `$error`: Verilator stops at the first `$error`, before the testbench can report.

7.2 Arbiter Assertions, Bound from `dv/sva/`

The arbitration rules are internal to `arb_mem`, so they are written in a checker module and attached with `bind`, as in Lab V1:

```
// hw/ip/arb_mem/dv/sva/arb_mem_sva.sv
module arb_mem_sva (
  input logic       clk, rst_n,
  input logic [1:0] req, gnt          // internal packed vectors of arb_mem
);
  default clocking @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_gnt_onehot0: assert property ($onehot0(gnt));            // A1
  a_gnt_req:     assert property ((gnt & ~req) == 2'b00);    // grant only if requested
  for (genvar i = 0; i < 2; i++) begin : g_port
    a_single: assert property (req[i] && !req[1-i] |-> gnt[i]);   // A2
    a_fair:   assert property (req[i] && !gnt[i]   |=> gnt[i]);   // A3
  end
  c_both_req:   cover property (req == 2'b11);
  c_alternate:  cover property (gnt == 2'b01 ##1 gnt == 2'b10 ##1 gnt == 2'b01);
  c_wait_one:   cover property (req[1] && !gnt[1] ##1 gnt[1]);
endmodule

// hw/ip/arb_mem/dv/sva/arb_mem_bind.sv
bind arb_mem arb_mem_sva u_sva (.clk(clk), .rst_n(rst_n), .req(req), .gnt(gnt));
```

Rule A4 needs one more assertion: keep a flag that is cleared by reset and set by the first grant, and require `gnt == 2'b01` for a conflict while the flag is still clear.

The fairness property `a_fair` is only meaningful because a waiting request is held (P2): its premise is guaranteed by `a_req_stable`. Properties that rely on each other are normal; write the dependency into the testplan. If your Lab R3 RTL uses different internal names, connect them explicitly in the bind file; do not rename RTL signals for the testbench.

7.3 Cover Properties

TEST TERM | Coverage (cover properties)

Not a test but a measurement: a cover property counts how often a scenario actually happened. A passing regression in which `c_both_req` was never hit has not tested the arbiter at all. The pass action `tb_utils_pkg::cover_hit()` prints `COVER:` lines that `make regress` sums over all runs and flags with `NOT HIT`.

At least: both masters request in the same cycle; grants alternate 0, 1, 0 in consecutive cycles; a write and a read to the same address from different masters in consecutive cycles; a master waits one cycle. The cross-port cover can be written in the bound checker, connecting the interface signals through the DUT's interface ports in the bind (`.m0_addr(m0.req_addr)`), or in a small checker instantiated in `tb_top` on `m_if[0]` and `m_if[1]`. The regression must hit every cover property.

SYNOPSYS CLOUD

With `make sim COV=1` VCS also records the assertion and cover metrics (`COV_METRICS` contains `assert`); `make cov` then lists every assertion and cover property with its attempt, success and failure counts in the URG report — an independent check of the `COVER:` lines. Optionally, prove the protocol and arbiter properties formally with VC Formal (`vcf`, part of VC Static): compile `arb_mem` with the bound checker, turn `a_req_stable` into an `assume property` for the slave's proof (it is the master's obligation) and let the tool prove `a_rsp_after_read`, `a_no_spurious_rsp`, `a_gnt_onehot0` and `a_fair` for all input sequences, not only the simulated ones.

7.4 Exercise: Assertion Fires First

TEST TERM | Negative / error-injection test

A bug is inserted on purpose to prove that the checkers can fail and to see *which* checker fails first. A checker that has never failed proves nothing. Never commit the broken RTL: work in a scratch copy, compiled through a scratch file list.

As in Lab V1, introduce one arbiter bug in a throw-away branch (or use the coordinator's buggy version), for example: the round-robin pointer never updates; master 1 has fixed priority; a read accepted in the same cycle as a write from the other master. Run `conflict` and `random` and record which assertion fires first, at which time, and what the scoreboard reports.

Pay attention to the first example: when the round-robin pointer never updates, every read still returns the correct data. *The data scoreboard never fails.* Only the fairness assertion and the latency statistics reveal the bug — or, if one master starves completely, the watchdog, hundreds of thousands of cycles later. A scoreboard checks *what* a design computes; assertions check *how* it behaves. Discuss this in `doc/verification.md`.

NOTE | If you find a real bug.

Fix it in `hw/ip/arb_mem/rtl/` with a separate commit whose message names the failing test, configuration and seed, add it to the bug log in `doc/verification.md`, and confirm that the Lab R3 smoke test and `make check-all` still pass.

8 Clocking Blocks (Optional)

Lab R2 established the drive standard: drive with `<=` right after the rising edge, sample at the rising edge. SystemVerilog has a construct that states the same timing in one place, the **clocking block**. It belongs in the interface, next to the signals it times, and is simulation-only:

```
`ifndef SYNTHESIS
  clocking mst_cb @(posedge clk);
    default input #1step output #0;
    output req_valid, req_we, req_addr, req_wdata;
    input  req_ready, rsp_valid, rsp_rdata;
  endclocking
  modport tb_master (clocking mst_cb, input clk, rst_n);
`endif
```

- **Input skew `#1step`:** inputs are sampled in the Preponed region, just before the edge — the same values concurrent assertions and flip-flops see.
- **Output skew `#0`:** drives through the clocking block (`vif.mst_cb.req_valid <= 1'b1;`) are applied after the DUT has sampled the edge (in the Re-NBA region), so the DUT sees them at the next edge — the same effect as the `<=` standard. A positive skew (`output #1`) delays the change further, which makes waveforms easier to read.
- **Cycle delays:** `@(vif.mst_cb)` waits for the next clocking event, and `##n` waits `n` clocking events.

```
@(vif.mst_cb);
vif.mst_cb.req_valid <= 1'b1;
vif.mst_cb.req_addr  <= seq[i].addr;
do @(vif.mst_cb); while (!vif.mst_cb.req_ready);
vif.mst_cb.req_valid <= 1'b0;
```

Clocking blocks remove the need for every driver author to understand the scheduling regions, which

is why UVM components (Lab T-DV2) commonly use them. Using one in this lab is optional; if you do, use it for every drive of that port — never mix clocking-block drives and direct `<=` drives of the same signal — and check that your Verilator version supports clocking blocks.

9 Testplan and Regression

Write `hw/ip/arb_mem/dv/testplan.md` before the tests, in the format of Lab V1. It has at least one row per protocol rule (P1–P5) and arbitration rule (A1–A4), plus the features "read-after-write across masters" and "all parameter sets". Each row names its tests, assertions and cover items.

TEST TERM | Regression

> The whole set of tests × seeds × parameter sets, run by one command (`make regress`) with a pass/fail summary and the summed cover counts. It is run before every merge; a *nightly* regression uses fresh seeds (`REGRESS_SEED_BASE`). Here it is 4 tests × 5 seeds × 3 address widths = 60 runs.

The regression uses `flow/mk/regress.mk` from Lab V1 unchanged and runs all four tests with 5 seeds on three address widths:

```
# hw/ip/arb_mem/dv/regress.list
config aw2     PARAMS="AW=2 DW=32"
config aw4     PARAMS="AW=4 DW=32"
config aw6     PARAMS="AW=6 DW=32"
test single      5  *
test write_read  5  *
test conflict    5  *    +IDLE_PCT=0 +ADDR_RANGE=2
test random      5  *    +NUM_TXN=5000
```

`AW=2` has only four words, so conflicts and read-after-write across masters are frequent; `AW=6` exercises the address decoding. If `regress.mk` cannot express something you need, fix it generically and confirm that the `seq_mul` and `sync_fifo` regressions still pass.

10 Requirements / Tasks

1. Write `dv/testplan.md` for `arb_mem`.
2. Build `dv/tb/tb_top.sv` with an array of two `mem_if` instances, an array of virtual interface handles and the DUT, parameterized by `AW` and `DW`.
3. Write `gen_seq`, `run_master`, `mon_req` and `mon_rsp` in `dv/env/arb_mem_env.svh`; every task that touches the DUT does so only through its `vif` argument.
4. Use a dynamic array per master for the sequence and the latencies, an array of two queues for expected data and an associative array for the reference model; justify each choice.
5. Start both drivers in parallel, the four monitors in the background and a watchdog that does not kill the monitors; end the test cleanly with empty `exp_q` queues.
6. Implement the four tests and four plusargs of the tables; report per-master statistics.
7. Add the four protocol assertions to `mem_if` and the arbiter assertions and cover properties in `dv/sva/`, bound to `arb_mem`; every cover property is hit.
8. Pass `make regress` for `AW=2`, `4` and `6` with `DW=32`, all tests, 5 seeds.
9. Complete the assertion-fires-first exercise, including the round-robin bug.
10. Optional: drive one master through a clocking block and show identical results for the same seed.

11 Repository and Flow

```
hw/ip/arb_mem/Makefile                  TB_TOP=tb_top, TB_FLIST=dv/dv.f, include regress.
    mk
hw/ip/arb_mem/rtl/mem_if.sv             changed: protocol assertions (and optional
                                        clocking block) inside `ifndef SYNTHESIS
hw/ip/arb_mem/dv/testplan.md
hw/ip/arb_mem/dv/regress.list
hw/ip/arb_mem/dv/dv.f
hw/ip/arb_mem/dv/tb/tb_top.sv
hw/ip/arb_mem/dv/env/arb_mem_env.svh
hw/ip/arb_mem/dv/tests/arb_mem_tests.svh
hw/ip/arb_mem/dv/sva/arb_mem_sva.sv
hw/ip/arb_mem/dv/sva/arb_mem_bind.sv
hw/ip/arb_mem/doc/verification.md       new: results, statistics, bug log
```

```
// hw/ip/arb_mem/dv/dv.f  (TB_FLIST: the RTL list first, then the testbench)
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.f
+incdir+${REPO_ROOT}/hw/ip/arb_mem/dv/env
+incdir+${REPO_ROOT}/hw/ip/arb_mem/dv/tests
${REPO_ROOT}/dv/common/tb_utils_pkg.sv
${REPO_ROOT}/hw/ip/arb_mem/dv/sva/arb_mem_sva.sv
${REPO_ROOT}/hw/ip/arb_mem/dv/sva/arb_mem_bind.sv
${REPO_ROOT}/hw/ip/arb_mem/dv/tb/tb_top.sv
```

No new make targets or variables are introduced; this lab uses the Lab L0 and Lab V1 interface unchanged. The default `TEST` of the Makefile must suit the selected testbench (`all` for the R3 smoke test, `random` here), for example `TEST ?= $(if $(filter tb_arb_mem,$(TB_TOP)),all,random)`, so that both command lines below work without `TEST=`. Example commands:

```
make -C hw/ip/arb_mem sim TEST=conflict SEED=3 PLUSARGS="+IDLE_PCT=0 +ADDR_RANGE=2"
make -C hw/ip/arb_mem sim PARAMS="AW=2 DW=32" TEST=random WAVES=1
make -C hw/ip/arb_mem sim TB_TOP=tb_arb_mem TB_FLIST=tb/tb.f      # R3 smoke test
make -C hw/ip/arb_mem regress
make check-all
```

SYNOPSYS CLOUD

All commands run unchanged with VCS (`SIM=vcs`, the cloud default); `make regress` passes `SIM` to every run. For debugging use `make sim TEST=conflict WAVES=1` and `make waves`: Verdi shows the interface instances `m_if[0]` and `m_if[1]` in the hierarchy (a virtual interface handle is a variable, not hierarchy, so you look at the instance it points to), the bound checker as `dut.u_sva` with its `req` and `gnt` ports. `$display` output of the monitors carries `%t` time stamps, so a failing compare can be found in the waveform directly.

12 Deliverables

One merge request from branch `lab/v2-arb-mem-dv`, tagged `v2-done` after review, containing the files above. `hw/ip/arb_mem/doc/verification.md` must contain:

- The array choices in the testbench: which kind for which data, and why.
- The process structure of the testbench: which processes exist, how they are started, how the test ends, and what `disable fork` kills.
- The shared-data argument: why the two request monitors may both write `ref_mem`, and which

assertion guarantees it.

- The regression summary and the per-master statistics of the `conflict` and `random` tests.
- The assertion and cover lists with cover counts, and the testplan status.
- The assertion-fires-first exercise, including the discussion of the bug the scoreboard cannot see.
- The bug log (real RTL bugs, failing test, seed, fix commit).

The timing diagram of a conflict made in Lab R3 (`doc/design.md`) is compared with a waveform of the `conflict` test; differences are either a documentation fix or a bug.

13 Assessment

Criterion	Weight
Functional verification: all tests and the regression pass, statistics reported	20%
Testbench processes: `fork/join` use, virtual interfaces, clean termination	20%
Arrays: correct choice and use of dynamic, queue and associative arrays	15%
Scoreboard in DUT acceptance order, unknown-read handling	15%
Protocol assertions in `mem_if`, arbiter assertions and cover properties	20%
Testplan, assertion-fires-first exercise and documentation	10%
Total	100%

14 Review Questions

1. Why can't you pass an interface instance directly to a task? Why can't you write `vif[i] = m_if[i]` in a loop?
2. Why must `req_valid` not depend on `req_ready`? What happens if both depend on each other combinationally?
3. Your two request monitors both write `ref_mem`. Why is that not a race here, and which assertion guarantees it?
4. What happens if you replace `join_any` with `join` in the watchdog pattern?
5. What does `disable fork` kill in your testbench? Is anything killed that should keep running?
6. Why does the scoreboard follow the DUT's acceptance order instead of the generated sequence order? Give a concrete sequence where the generated order gives the wrong expected data.
7. Why did you choose a dynamic array for the sequence, a queue for the expected data and an associative array for the memory model?
8. The round-robin pointer is stuck, yet every test passes the scoreboard. Which checks find the bug?
9. `a_req_stable` fires. Is the bug in the DUT or in the testbench, and how do you tell?
10. What does a clocking block's `#1step` input skew have in common with how concurrent assertions sample?

15 References

- IEEE Std 1800-2017: Clause 7 (aggregate data types: arrays, queues, associative arrays and their methods), Clause 9 (processes, `fork/join`), Clause 14 (clocking blocks), Clause 16 (assertions), Clause 25 (interfaces, virtual interfaces).
- *SystemVerilog for Verification*, Spear and Tumbush: chapters on data types, connecting the testbench and design, and threads.
- Clifford E. Cummings, SystemVerilog Event Regions, Race Avoidance and Guidelines, SNUG 2006.
- OpenTitan, Design Verification Methodology (testplans, protocol assertions in interfaces).
- Verification Academy: SystemVerilog testbench material on interfaces, processes and assertions.
- Verilator User's Guide: `--timing`, supported SystemVerilog constructs.